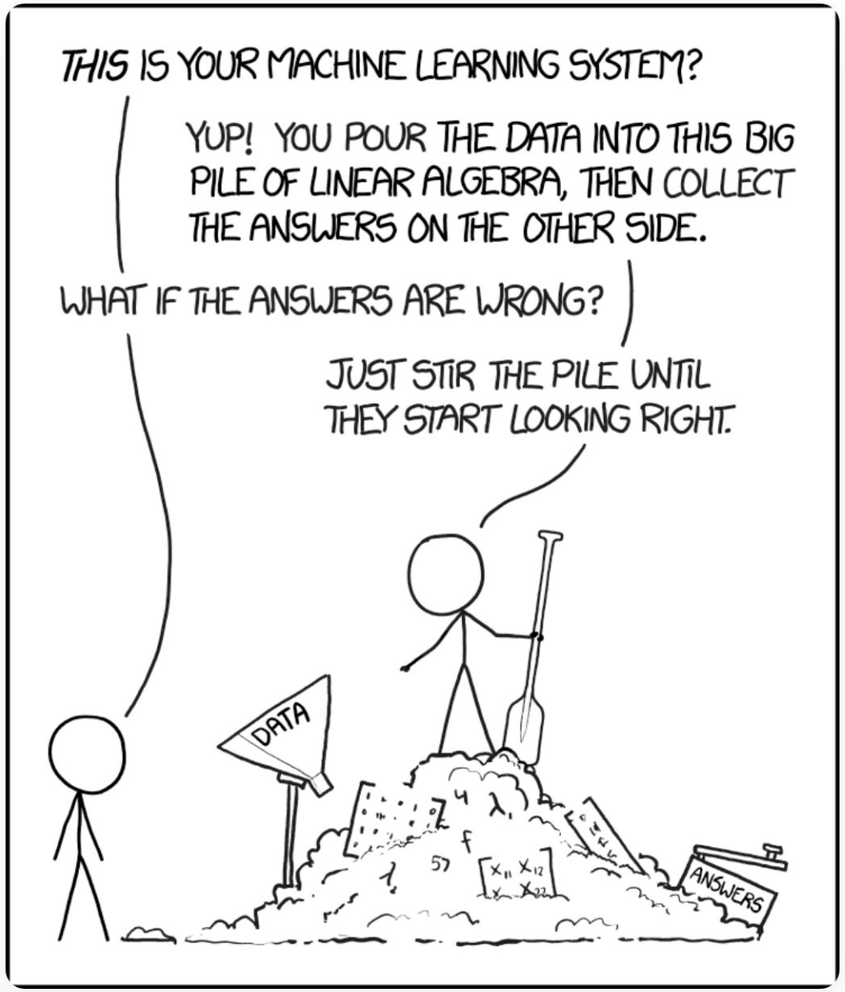
MACHINE LEARNING

Neural networks

Part 2: training a network.



"Machine Learning", xkcd 1838, by Randall Munroe (xkcd.com/1838). CC BY-NC 2.5.

LAST TIME

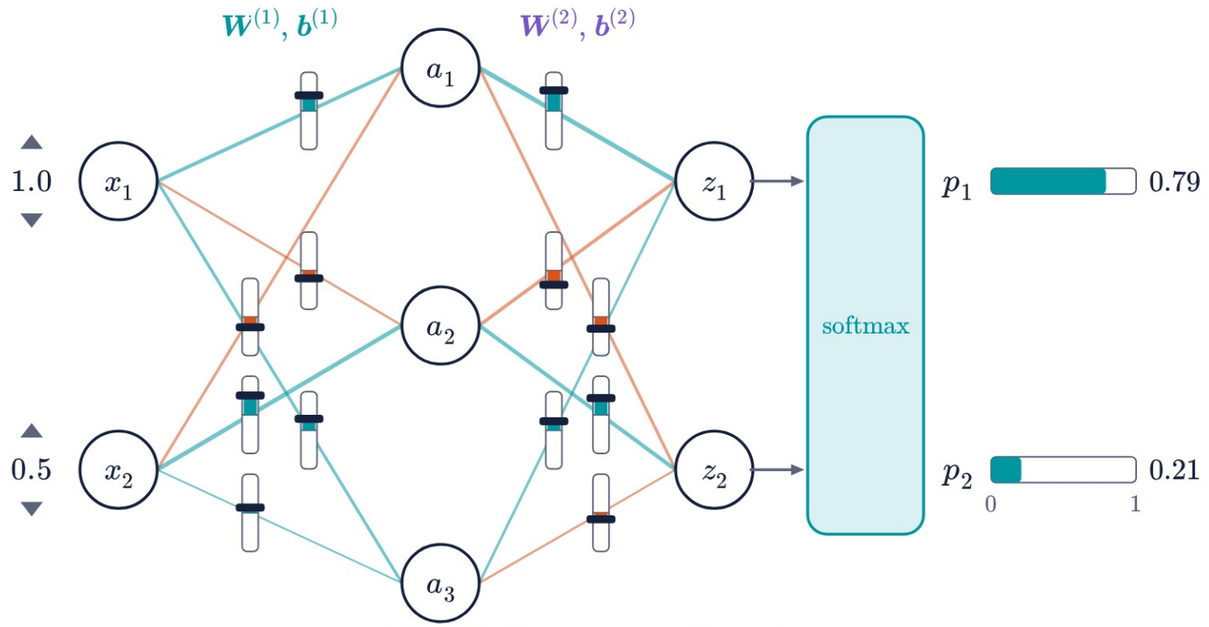
Forward propagation

$$
\begin{aligned}
\boldsymbol{a}^{(1)} &= \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}) \\
\boldsymbol{z} &= \boldsymbol{W}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)} \\
\boldsymbol{p} &= \mathrm{softmax}(\boldsymbol{z})
\end{aligned}
$$

- Train by gradient descent:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

- We need $\partial \mathcal{L} / \partial w$ for every weight w. Where does it come from?



The network is one function

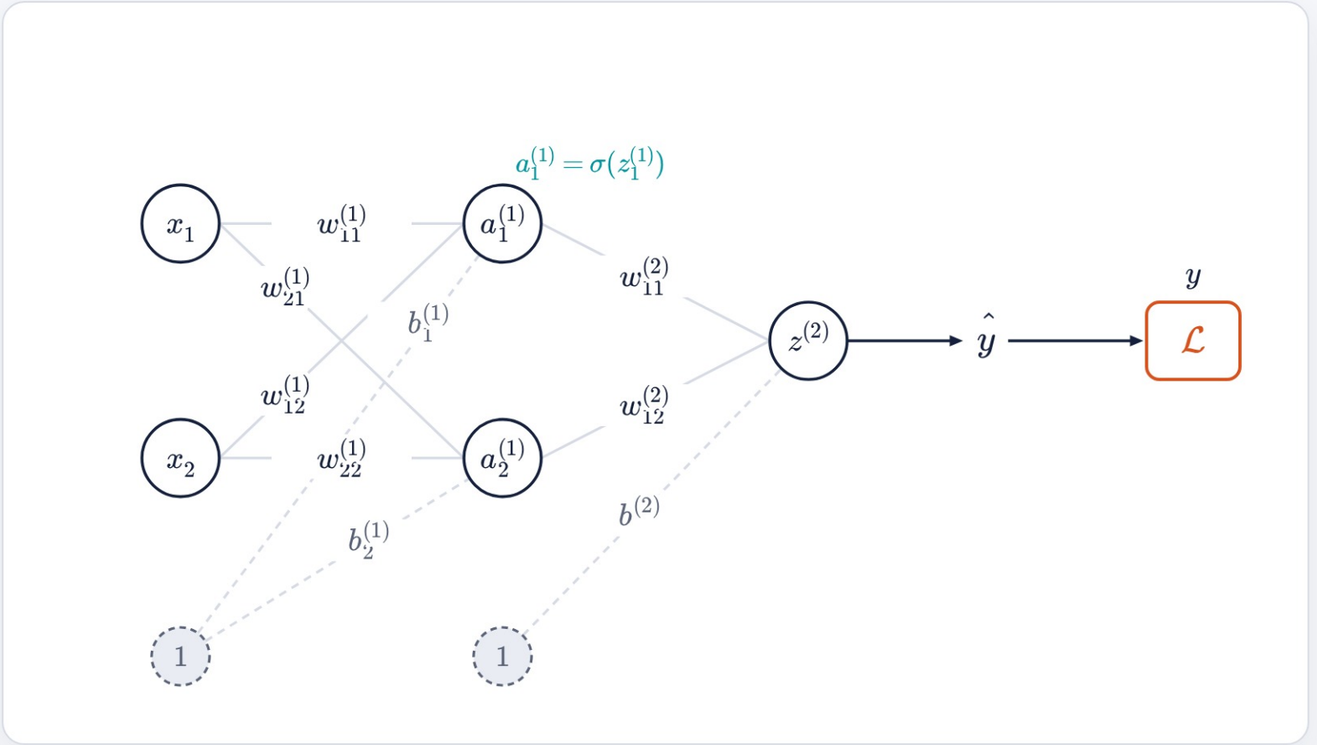
- Notation, for layer l: each unit computes a weighted sum $z^{(l)}$, the **pre-activation**, then its **activation** $a^{(l)} = \varphi(z^{(l)})$. The inputs are $a^{(0)} = \boldsymbol{x}$.

  $w_{ji}^{(l)}$ is the weight from unit i of layer $l-1$ to unit j of layer l; $b_j^{(l)}$ is unit j's bias.

- A small network for regression: two sigmoid hidden units, one linear output. Written out in full:

$$\hat{y} = w_{11}^{(2)}\,\sigma(w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)}) + w_{12}^{(2)}\,\sigma(w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)}) + b^{(2)}$$

  The two $\sigma(\cdot)$ terms are the hidden activations $a_1^{(1)}$ and $a_2^{(1)}$.

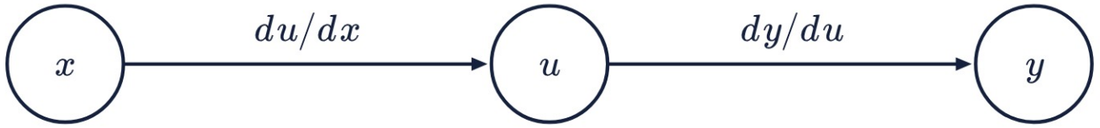


BACKPROPAGATION

The chain rule

- If y depends on u, and u depends on x, the derivatives multiply:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

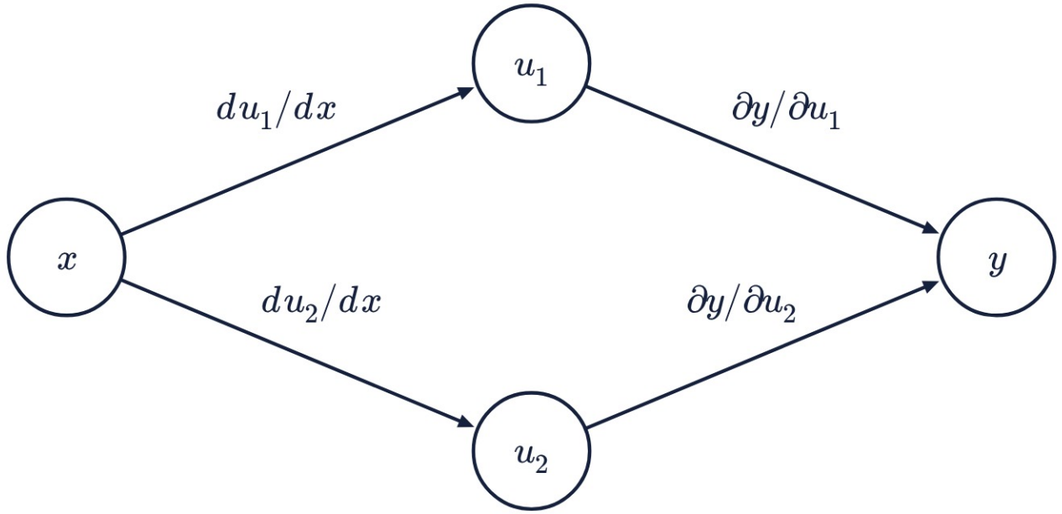


BACKPROPAGATION

When the chain branches

- If x affects y along two paths, through u_1 and through u_2, add the paths:

$$\frac{dy}{dx} = \frac{\partial y}{\partial u_1} \cdot \frac{du_1}{dx} + \frac{\partial y}{\partial u_2} \cdot \frac{du_2}{dx}$$



Derivatives we will use

- Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)\left(1 - \sigma(z)\right)$$

- Logarithm:

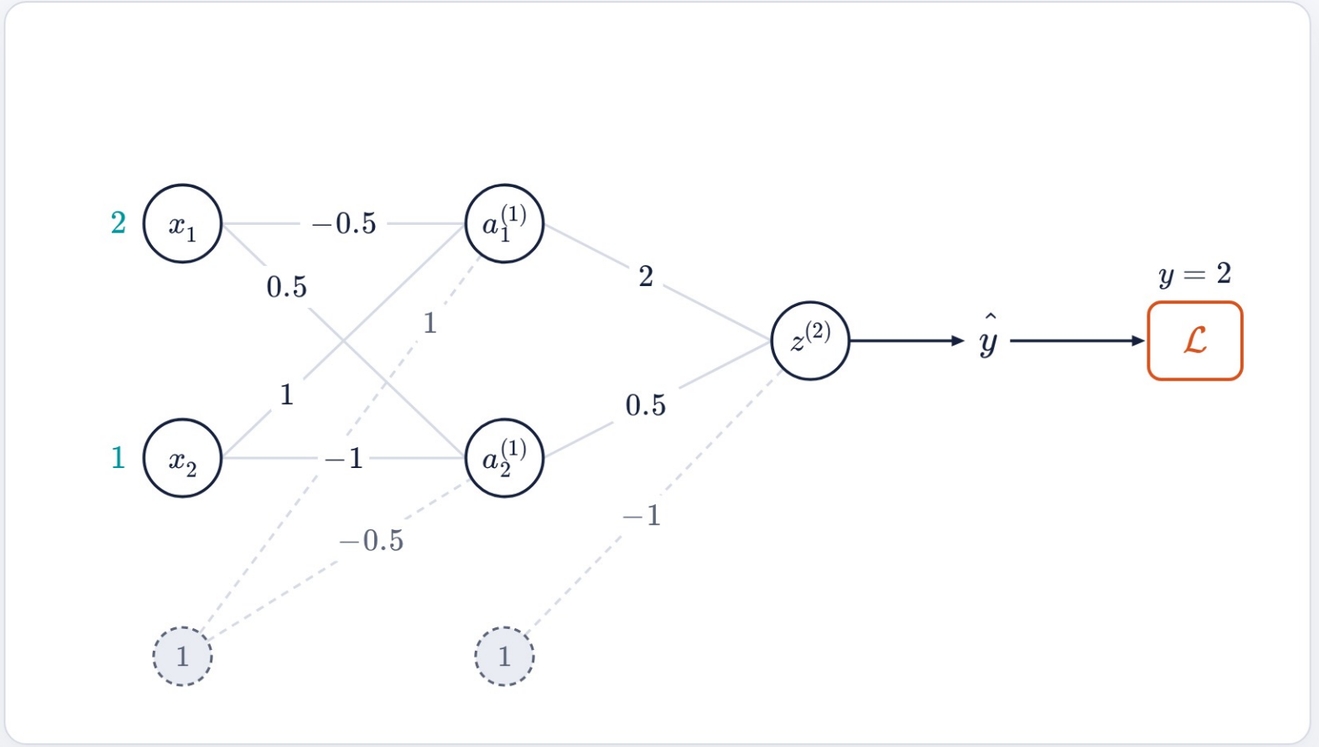
$$\frac{d}{dp} \log p = \frac{1}{p}$$

- Softmax followed by cross-entropy (last class):

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k$$

Regression example

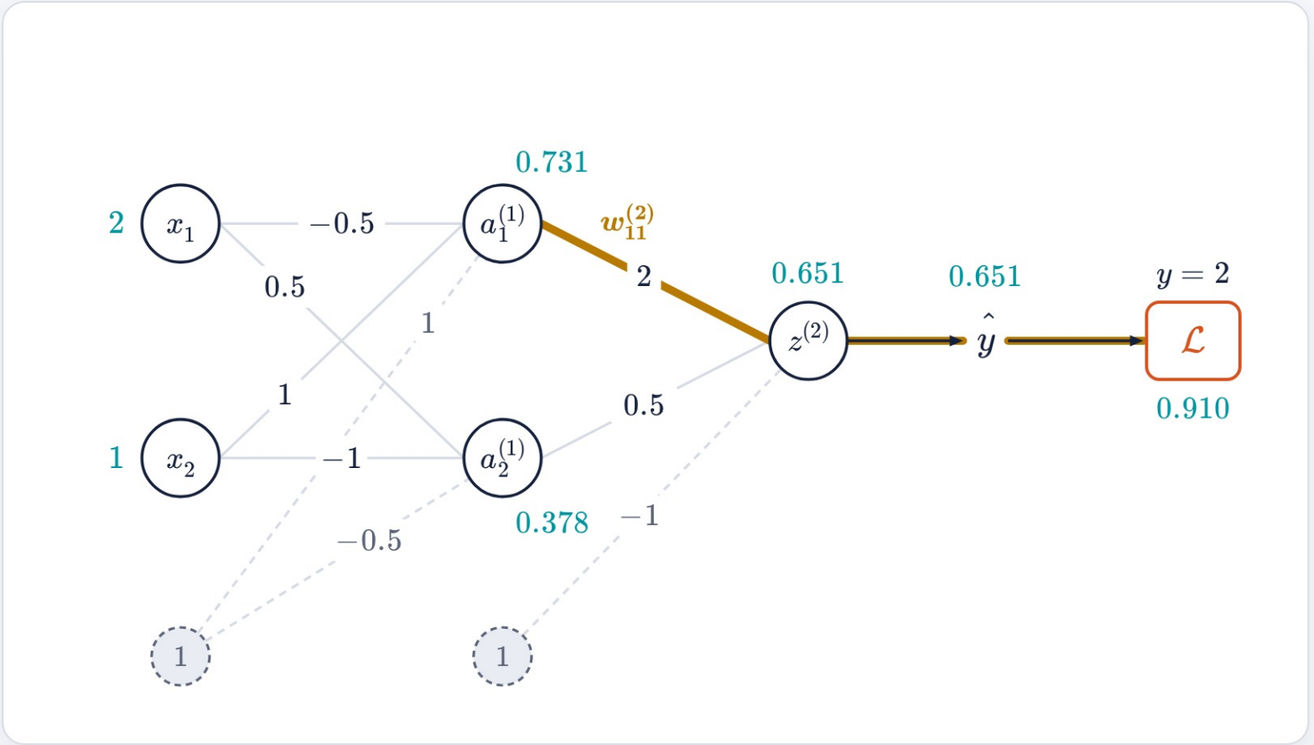
- One training point: $\boldsymbol{x} = (2, 1)$, target $y = 2$. The weights are on the connections. The output is linear: $\hat{y} = z^{(2)}$.



An output weight: $w_{11}^{(2)}$

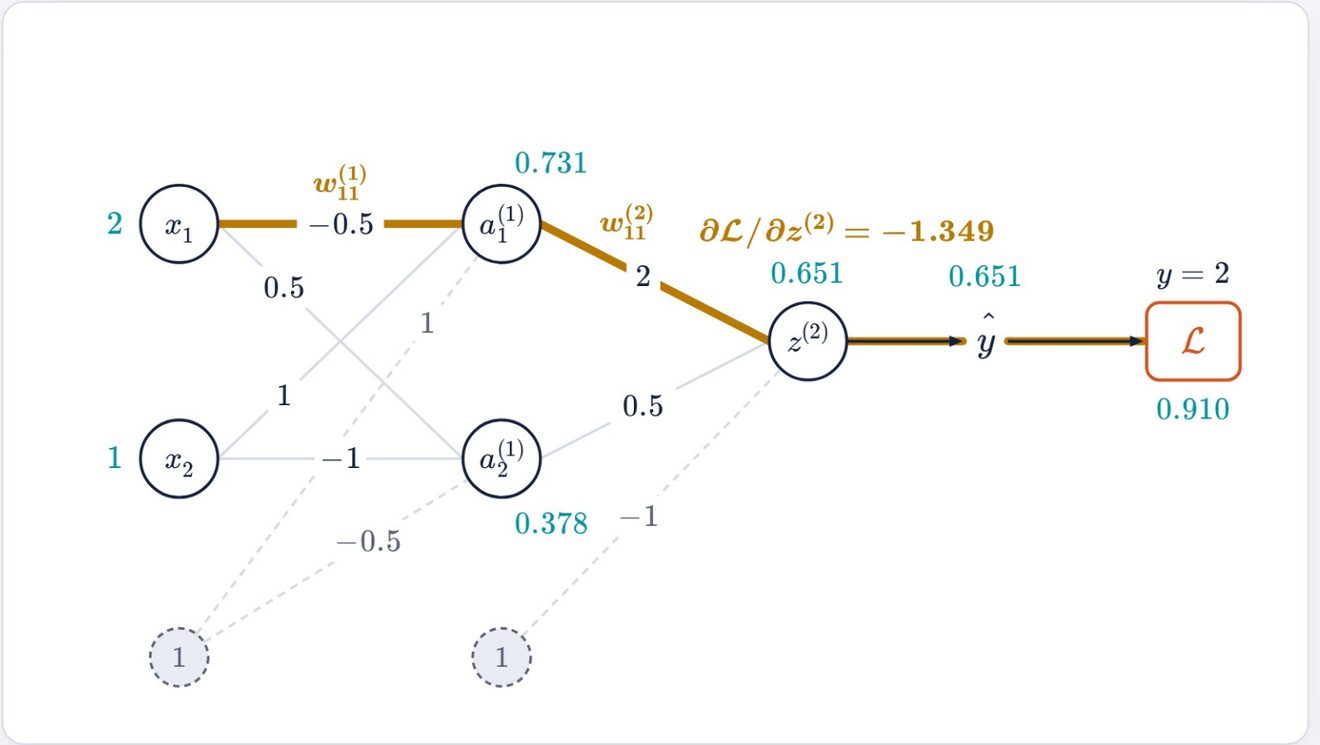
- $w_{11}^{(2)}$ changes the loss along the highlighted path, $w_{11}^{(2)} \to z^{(2)} \to \hat{y} \to \mathcal{L}$. The chain rule multiplies the derivatives along it:

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial w_{11}^{(2)}}$$



A hidden weight: $w_{11}^{(1)}$

- The path is longer:
  $w_{11}^{(1)} \to z_1^{(1)} \to a_1^{(1)} \to z^{(2)} \to \hat{y} \to \mathcal{L}$.
  $$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = \frac{\partial \mathcal{L}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \cdot \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$
- $\partial \mathcal{L}/\partial z^{(2)} = -1.349$, already computed for $w_{11}^{(2)}$. Reuse it.



BACKPROPAGATION

Update the weights

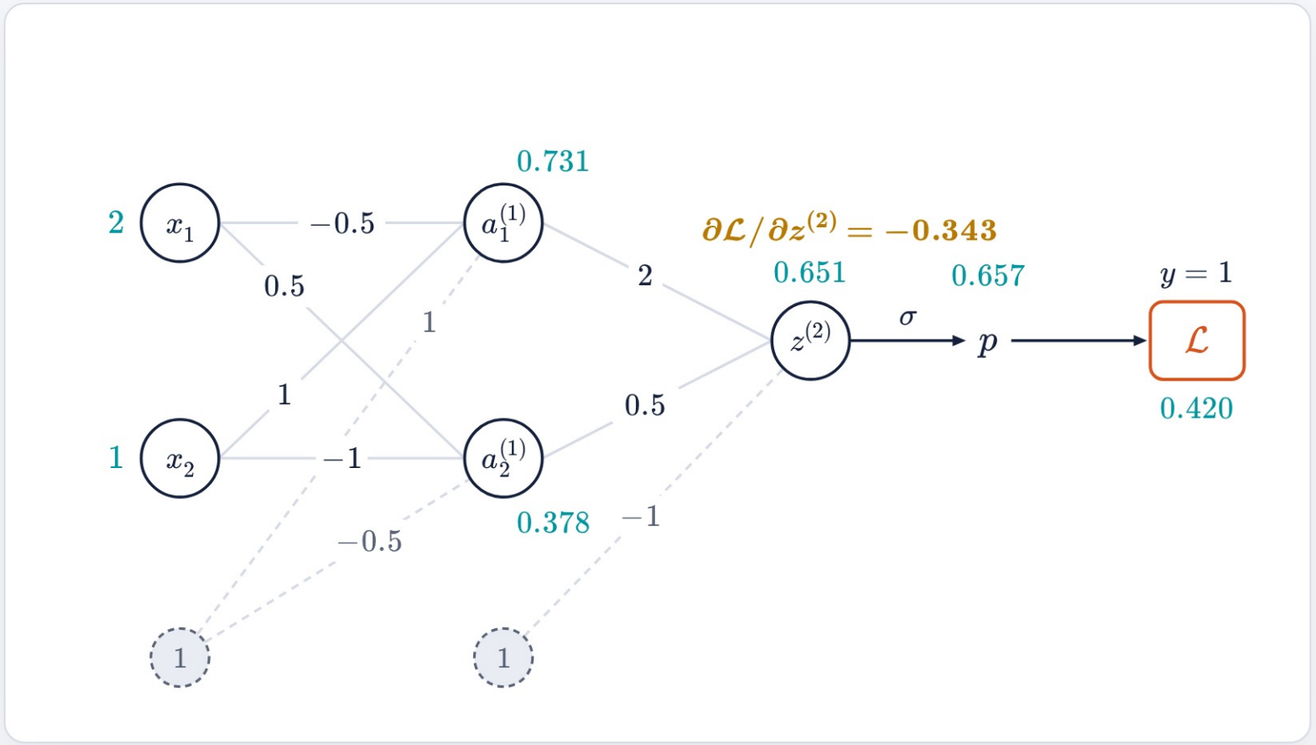
- Gradient descent, with learning rate $\eta = 0.1$:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$



Classification example

- Same network and point, but the target is now a class label, $y = 1$. Put a sigmoid on the output, so it is a probability: $p = \sigma(z^{(2)}) = \sigma(0.651) = 0.657$.
- The log loss, as for logistic regression:
$$\mathcal{L} = -y \log p - (1 - y) \log(1 - p)$$
$$= -\log 0.657 = 0.420$$
- Only the start of the chain changes. The sigmoid's slope cancels:
$$\frac{\partial \mathcal{L}}{\partial z^{(2)}} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z^{(2)}} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right) p(1 - p)$$
$$= p - y = 0.657 - 1 = -0.343$$
- The rest of the chain is the same as for regression:
$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(2)}} = -0.343 \times 0.731 = -0.251$$
$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = -0.343 \times 2 \times 0.1966 \times 2 = -0.270$$



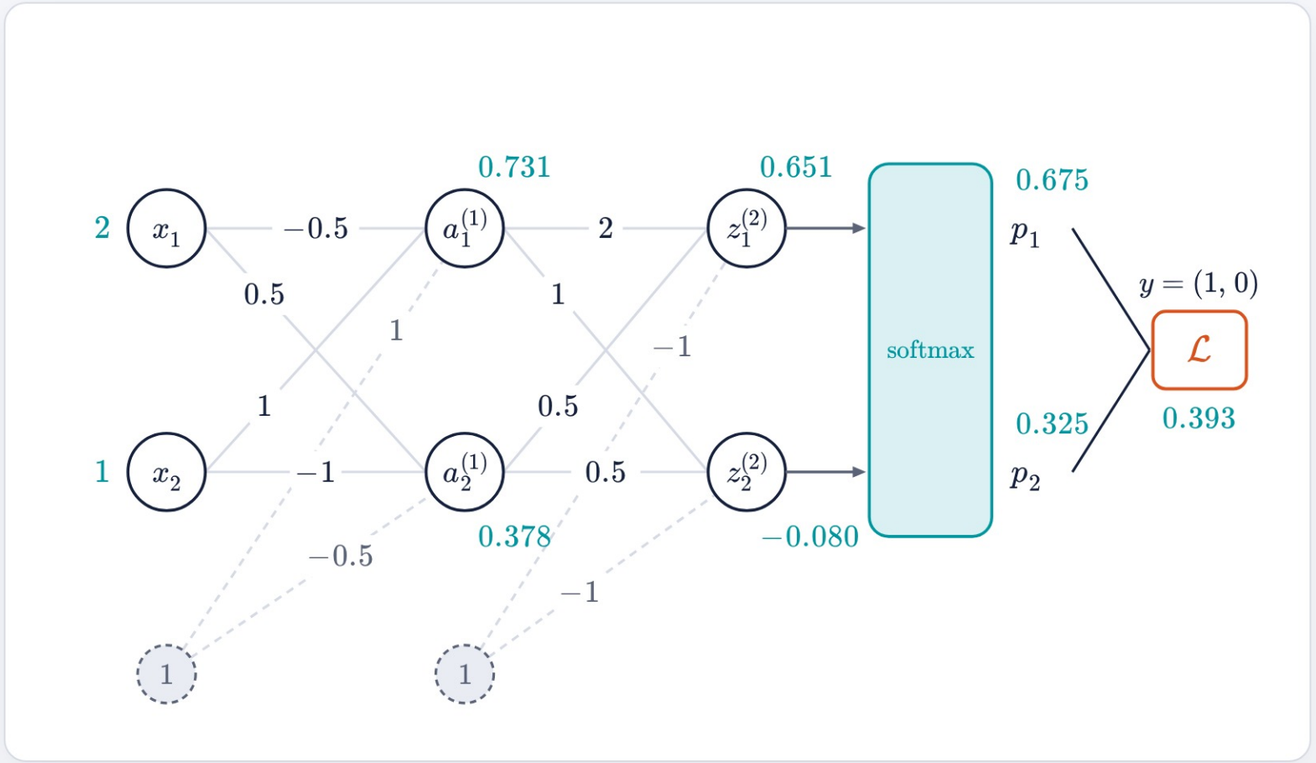
Two outputs

- Same input and first layer, so $a_1^{(1)} = 0.731$, $a_2^{(1)} = 0.378$. Now two outputs, a softmax and cross-entropy. The label is class 1: $\boldsymbol{y} = (1, 0)$.
  $w_{kj}^{(2)}$ is the weight from hidden unit j to output k.
- The output pre-activations:

$$z_1^{(2)} = 2 \cdot 0.731 + 0.5 \cdot 0.378 - 1 = 0.651$$
$$z_2^{(2)} = 1 \cdot 0.731 + 0.5 \cdot 0.378 - 1 = -0.080$$

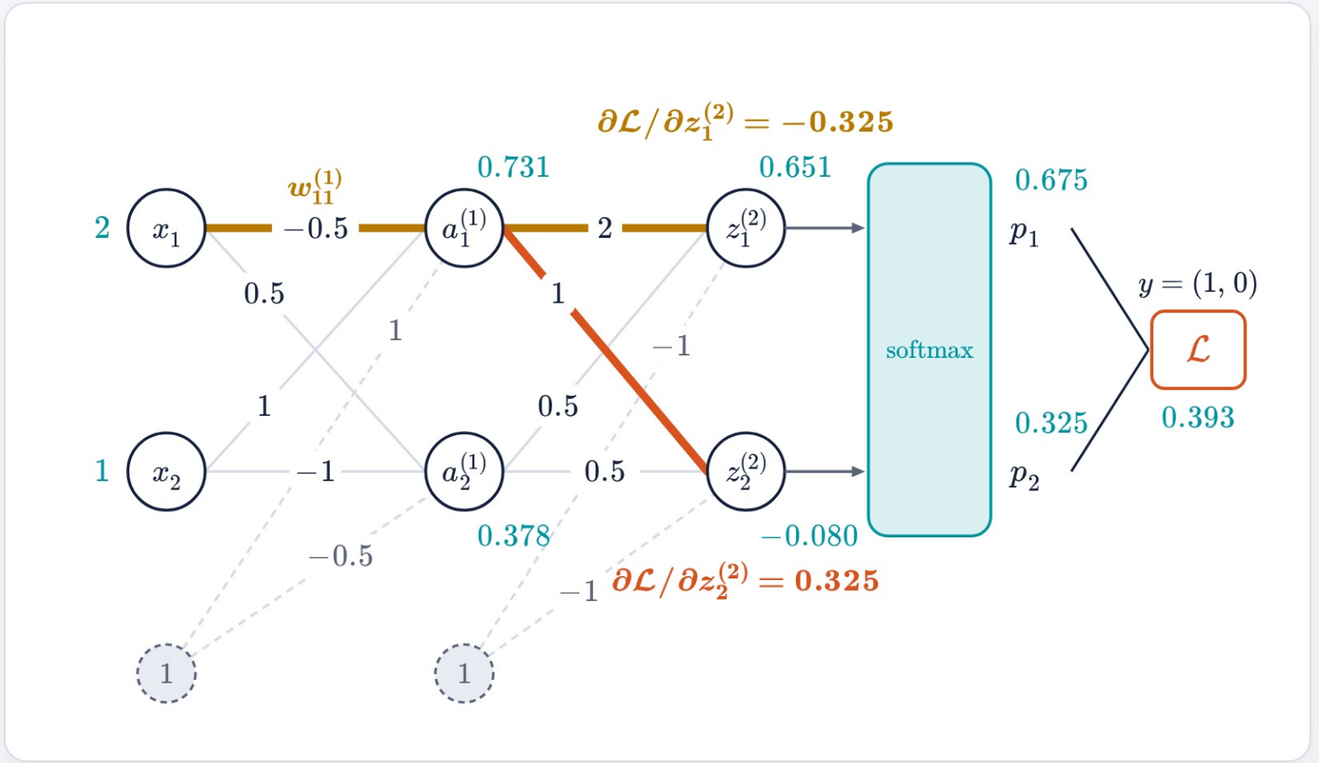
- The softmax and the loss:

$$p_1 = \frac{e^{0.651}}{e^{0.651} + e^{-0.080}} = \frac{1.917}{1.917 + 0.923} = 0.675$$
$$p_2 = 0.325, \qquad \mathcal{L} = -\log 0.675 = 0.393$$



Add the paths

- $w_{11}^{(1)}$ changes $a_1^{(1)}$, and $a_1^{(1)}$ feeds both outputs. So $w_{11}^{(1)}$ reaches the loss along two paths.



The general rule

- Every unit j of layer l works the same way:

  $$z_j^{(l)} = \sum_i w_{ji}^{(l)}\, a_i^{(l-1)} + b_j^{(l)}, \qquad a_j^{(l)} = \varphi(z_j^{(l)})$$

  "Error" means $\partial \mathcal{L}/\partial z$: how the loss changes with a unit's pre-activation.

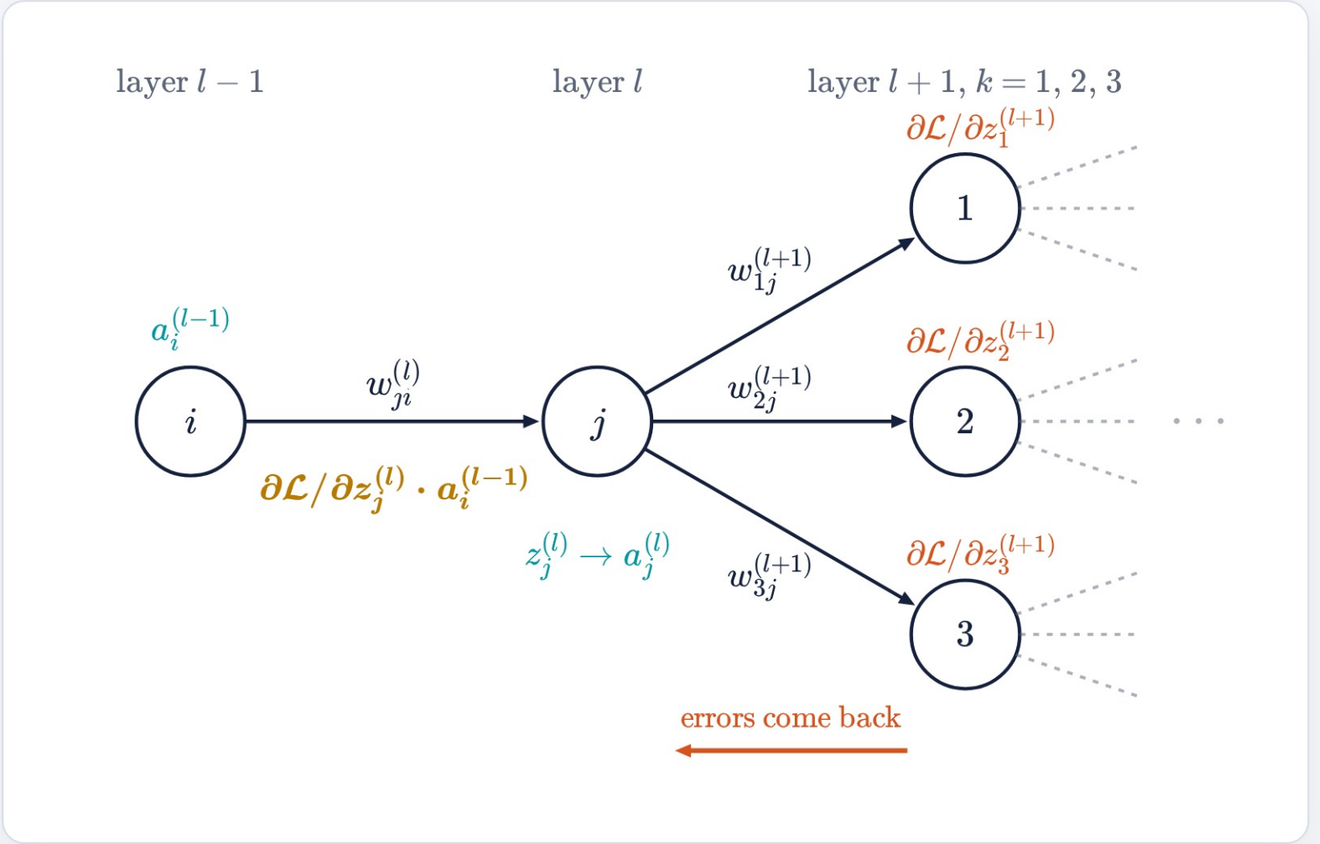
- **Any weight**: the error at its end times the activation at its start.

  $$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \cdot a_i^{(l-1)}$$

- **A hidden unit**: its slope times the errors of the units it feeds, each times its weight.

  $$\frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \varphi'(z_j^{(l)}) \sum_k w_{kj}^{(l+1)}\, \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}}$$

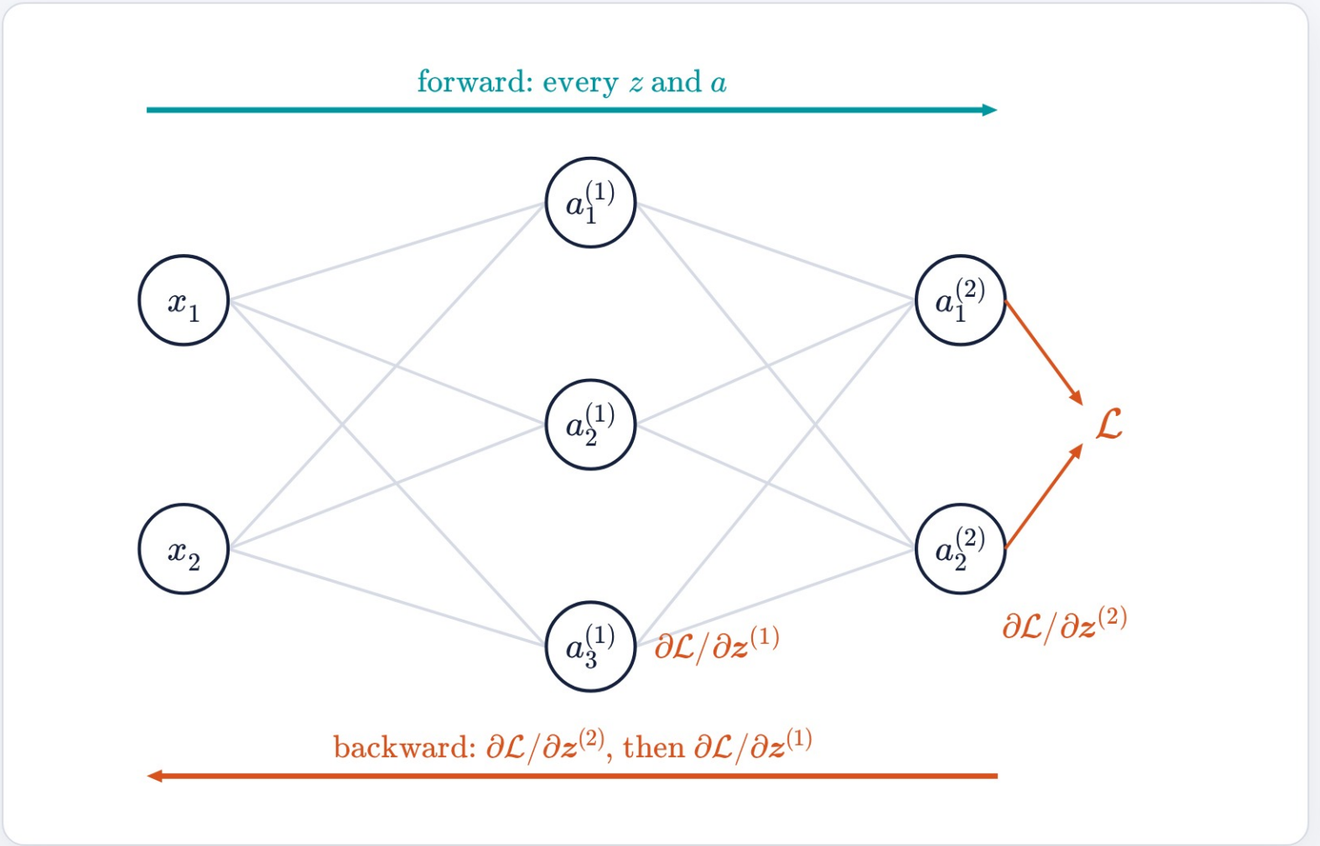
- **An output unit**: from the loss. $\hat{y} - y$ for regression with squared error; $p - y$ for sigmoid or softmax with cross-entropy.



BACKPROPAGATION

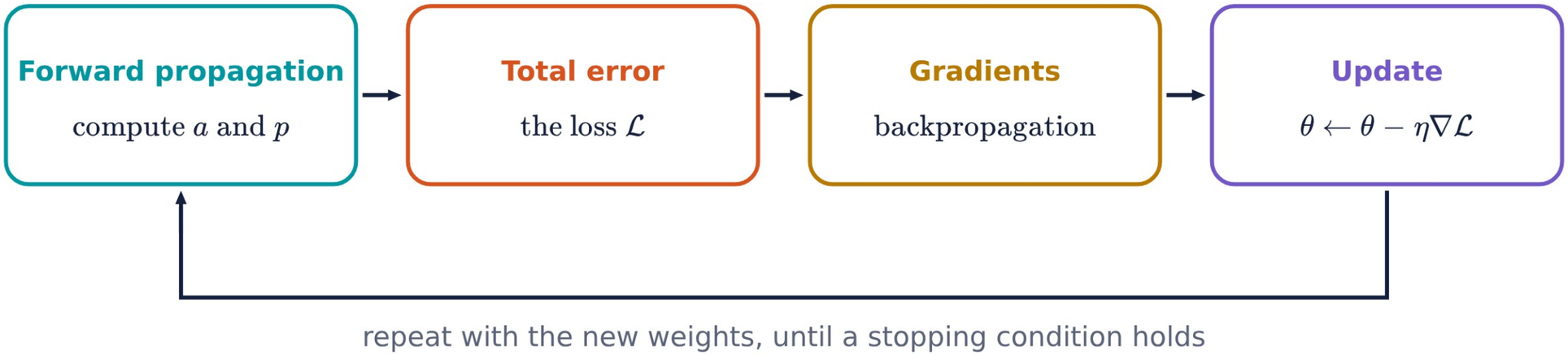
The algorithm

1. **Forward pass**: compute and store every z and a, layer by layer.
2. **Loss**: compute $\mathcal{L}$ from the output and the label.
3. **Backward pass**: start at the output with $\partial \mathcal{L} / \partial z_k^{(2)}$ (for softmax and cross-entropy, $p_k - y_k$). Then go back layer by layer with the hidden-unit rule.
4. **Gradients**: each weight gets the error at its end times the activation at its start.
5. **Update** every weight, $w \leftarrow w - \eta \, \partial \mathcal{L} / \partial w$, using the old weights throughout.



PUTTING IT TOGETHER

The training loop



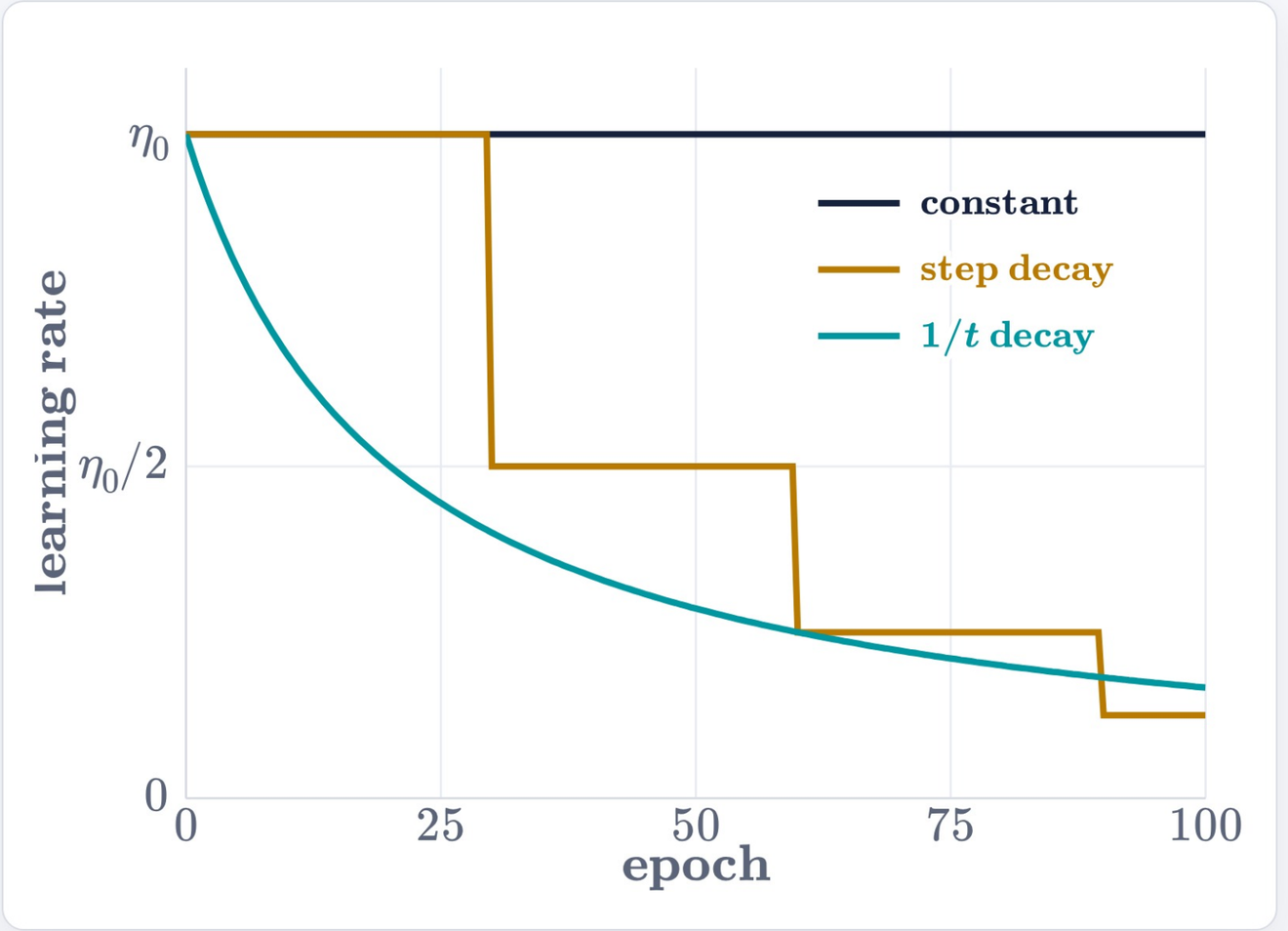
TRAINING IN PRACTICE

The learning rate

- The update:

  $$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$$

  Too small: slow. Too large: it overshoots. (Last class.)
- Large steps help early; small steps help near the end.
- A **learning-rate schedule** lowers η as training goes on.



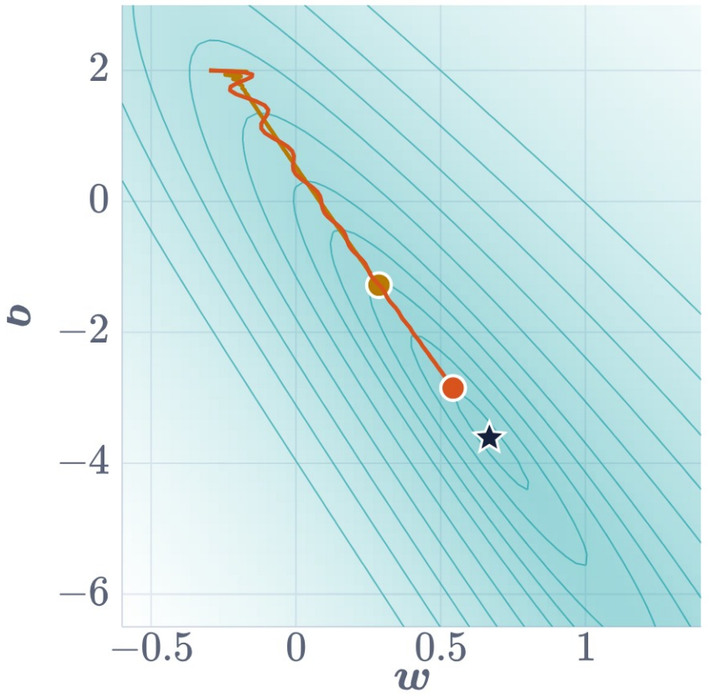
TRAINING IN PRACTICE

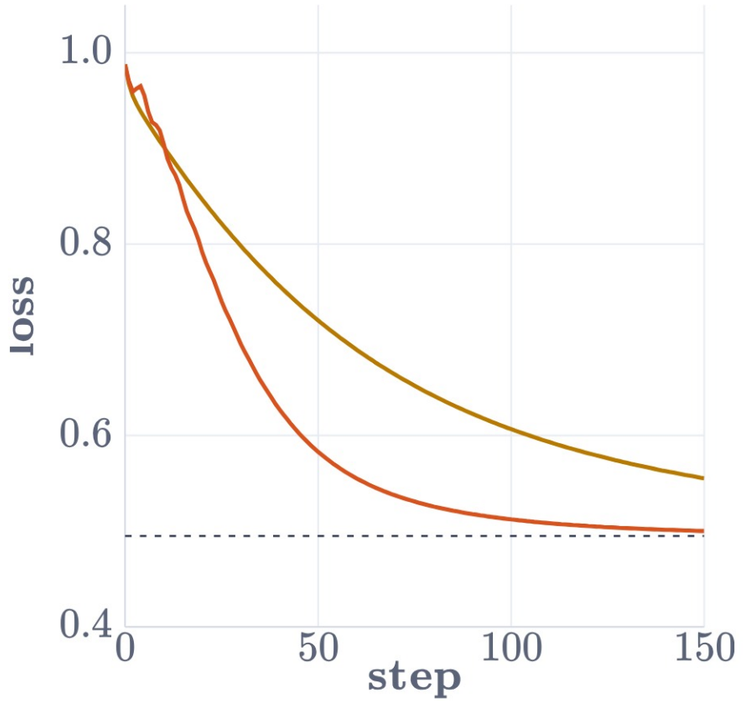
Momentum

- Keep a running sum of past gradients:

  $\boldsymbol{v} \leftarrow \beta \boldsymbol{v} + \nabla \mathcal{L}$

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{v}$





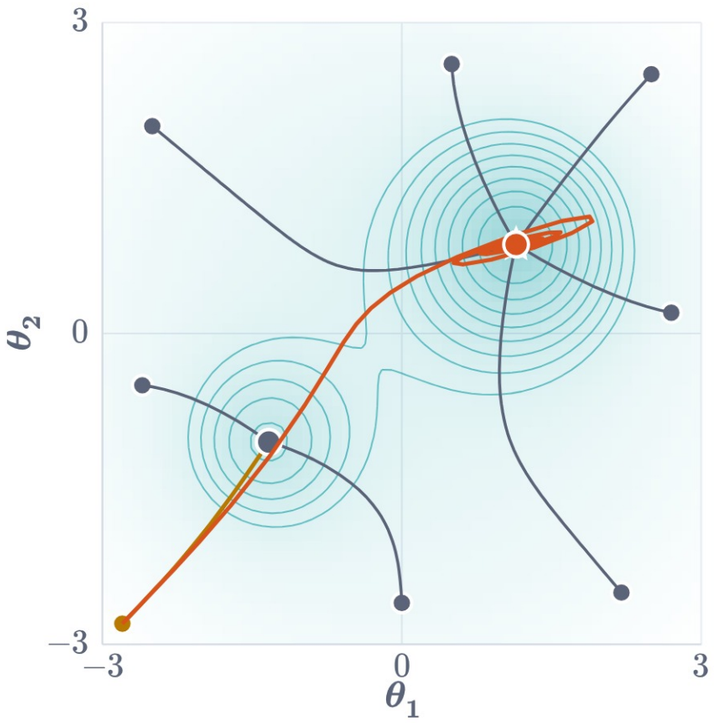
STEP	PLAIN LOSS	MOMENTUM LOSS
150	0.555	0.500

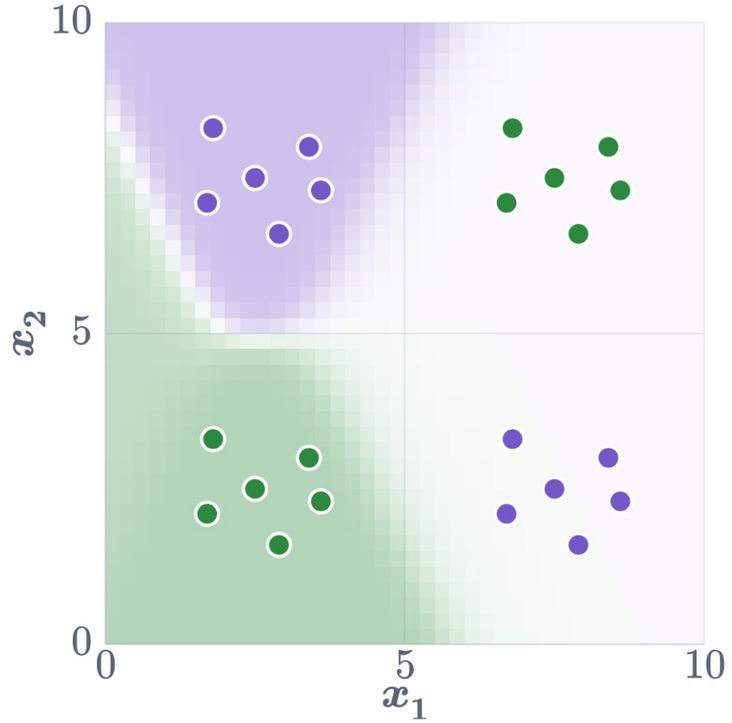
▶ step

TRAINING IN PRACTICE

Local minima

- Gradient descent stops in the valley it starts in.
- Restart from several starting points; keep the lowest loss.
- Momentum can roll through a shallow valley.
- It happens on real networks: the XOR network from a different start stalls.



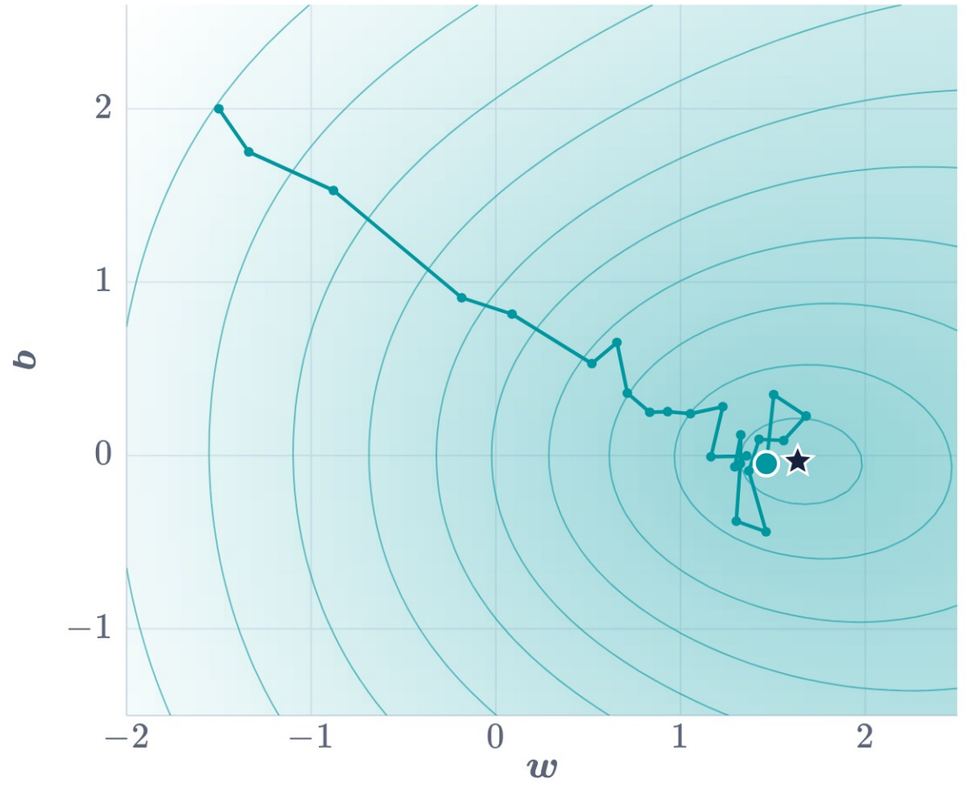


STEPS	LOSS	ACCURACY
4000	0.347	58%

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.
- **Stochastic**: from one point at a time. Many cheap, noisy steps.
- **Mini-batch**: from a few points, here 5. In between.
- An **epoch** is one pass through the training data.



POINTS PER STEP
5

STEPS PER EPOCH
4

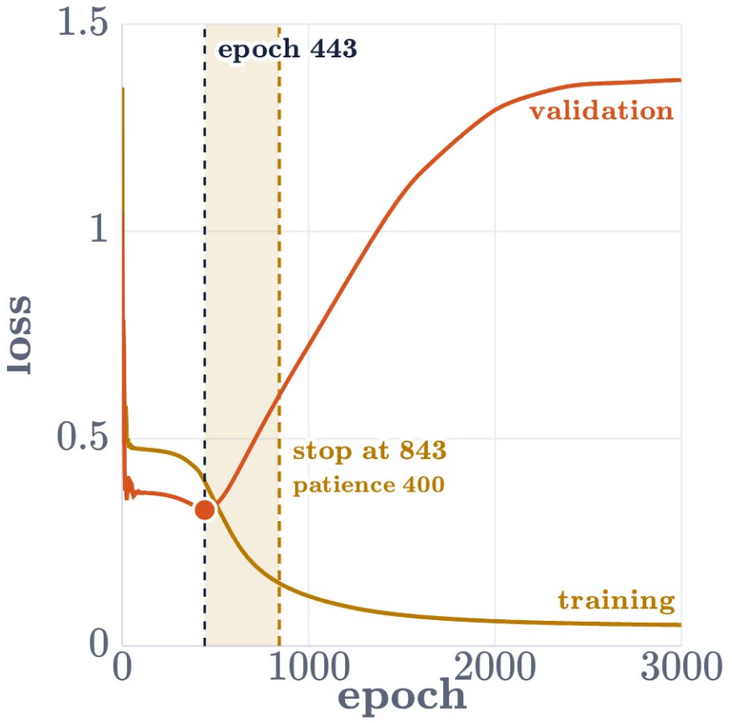
LOSS AFTER 6 EPOCHS
0.50

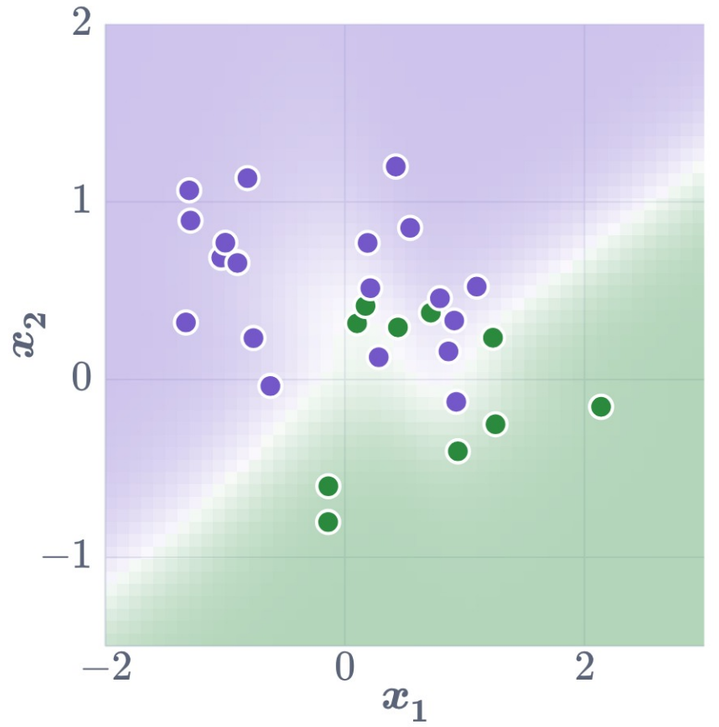
batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.
- The validation loss goes down, then up: the network starts to overfit.
- **Early stopping**: keep the weights from the epoch with the lowest validation loss.
- **Patience**: stop once the validation loss has not improved for a set number of epochs.





EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
443	80%	89%

SUMMARY

Neural networks: characteristics

Strengths

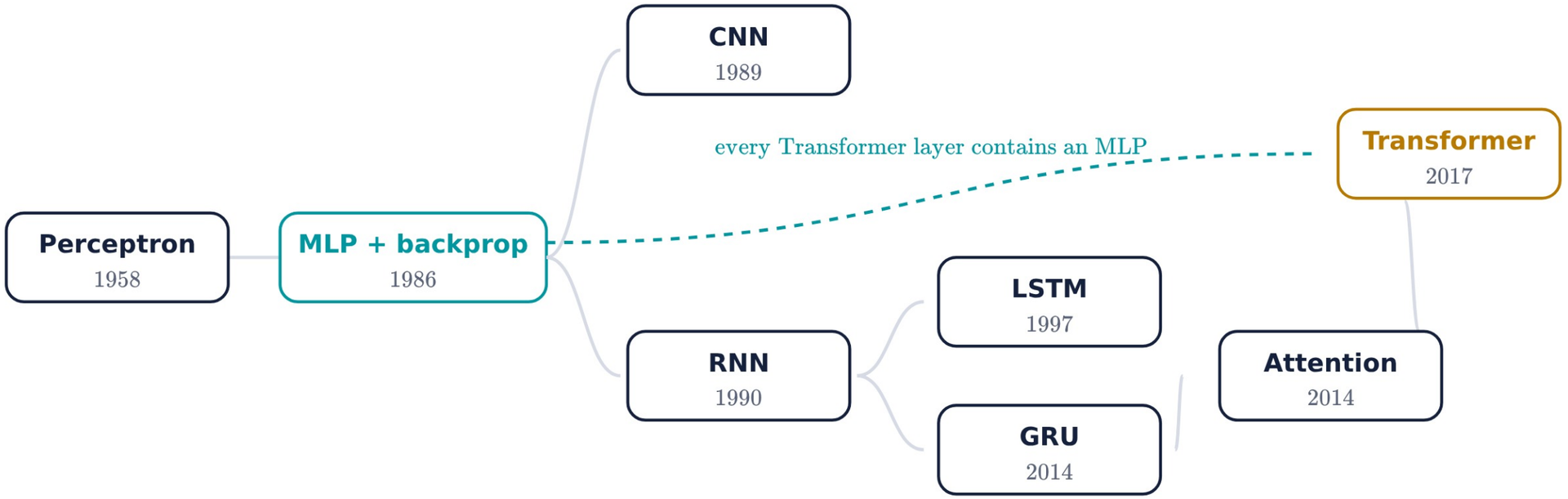
- Very flexible: one hidden layer with enough units can approximate any continuous function on a bounded region.
- They learn their own features, so less hand-made feature design.
- They improve with more data and computing power, and handle many outputs at once.
- Irrelevant inputs can be ignored: training can drive their weights to zero.

Weaknesses

- Non-convex training: the answer depends on the start, with no guarantee of the best weights.
- Many choices to tune: layers, units, η, optimizer, epochs.
- They need a lot of data, overfit small data sets, and are hard to interpret.
- Training is time-consuming: many passes over the data, and many runs to tune the choices above.

WHERE THIS GOES

The family tree



SUMMARY

Takeaways

Backpropagation

The chain rule, from the output back. At the output, $p_k - y_k$; a hidden unit adds up the paths through every output it feeds. Each weight's gradient is the error at its end times the value at its start.

Optimizers

Learning-rate schedules and momentum.

Batches

Batch, mini-batch, stochastic. An epoch is one pass through the data.

Stopping

Watch the validation loss; stop early.

Thank you.

Questions?

BONUS

Every path from one weight

- Three hidden layers of 3 units, two outputs. Which paths lead from $w_{11}^{(1)}$ to the loss?
- $w_{11}^{(1)}$ changes only $a_1^{(1)}$.
- $a_1^{(1)}$ feeds all 3 units of layer 2: 3 paths.
- Each of those feeds all 3 units of layer 3: $3 \times 3 = 9$ paths.
- Each of those feeds both outputs: $9 \times 2 = 18$ paths to $\mathcal{L}$. The chain rule adds 18 products.
- Backpropagation never lists the paths. The hidden-unit rule adds them up one layer at a time, so the work grows with the number of weights, not the number of paths.

